

Face Recognition on Extended Yale B Dataset

A Comparative Study of Classical and Deep Learning Classifiers

N. Rushil Krishna Sai N. Suhas P. Sanjana Trisham Bepari Ashish Kumar

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

CS Mini Project, 2024

Table of Contents

Classical ML

- ➊ Problem Statement & Dataset
- ➋ Data Preprocessing & PCA
- ➌ K-Nearest Neighbours
- ➍ Gaussian Naive Bayes
- ➎ Logistic Regression
- ➏ Support Vector Machine
- ➐ Random Forest
- ➑ Classical ML Summary

Deep Learning

- ➒ CNN from Scratch
- ➓ ResNet50 Transfer Learning
- ➔ FaceNet Metric Learning
- ➕ Deep Learning Comparison
- ➖ Conclusion

Problem Statement

Objective

Perform **face recognition** on the Extended Yale B dataset under **extreme illumination variation** and systematically compare classical ML vs. deep learning classifiers.

Core Challenges

- Illumination drastically alters facial appearance
- High-dimensional raw pixel space (32,256 dims)
- 28-subject multi-class recognition task
- Need for both accuracy and efficiency

Key Insights

- PCA + SVM is competitive for resource-constrained settings
- Deep learning outperforms classical methods via non-linear hierarchical features
- Domain-specific pretraining (FaceNet) beats general pretraining (ResNet50)
- Shadow recovery is critical for extreme lighting

Dataset: Extended Yale B

Property	Details
Total Subjects	28 (IDs 11–39, no 14)
Images / Subject	585
Total Images	16,380
Image Format	PGM (grayscale)
Resolution	168×192 pixels
Feature Dims	32,256 per image
Illumination	9 subsets
Class Balance	Perfectly balanced

Table: Train / Test Splits

Split	Samples	%
Classical Train	13,104	80%
Classical Test	3,276	20%
DL Train	11,466	70%
DL Val	2,457	15%
DL Test	2,457	15%

Key Insight

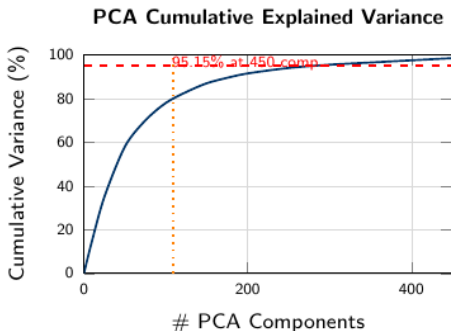
The same identity appears very different under extreme side or back lighting — this is what makes the dataset challenging.

Data Preprocessing & PCA

Preprocessing Pipeline

- 1 **Load & Flatten:** PGM images $\rightarrow$ 1D vectors ($168 \times 192 \rightarrow 32,256$ dims)
- 2 **Normalize:** Pixel values $\div 255 \rightarrow$ range $[0, 1]$
- 3 **Stratified Split:** 80/20 train-test, each class proportionally represented
- 4 **PCA Reduction:** 32,256 dims $\rightarrow$ 450 principal components

Components	450
Variance Retained	95.15%
Reduction Ratio	$\sim 71.7\times$
Top Eigenvalue	274.50



Why PCA?

Motivation

- 32,256 features $\Rightarrow$ curse of dimensionality
- Nearby pixels are highly correlated
- PCA decorrelates features & removes noise
- Focuses on maximum variance directions

Cumulative Variance

- Variance grows rapidly in the first 100 components
- 450 components capture **95.15%** variance

Sensitivity Analysis

Components tested: {100, 150, 300, 450}. **450** consistently achieved the best accuracy–complexity tradeoff, with negligible gain beyond.

Denoising Benefit

By discarding the least significant components (which often encode noise), PCA produces smoother, more generalizable representations.

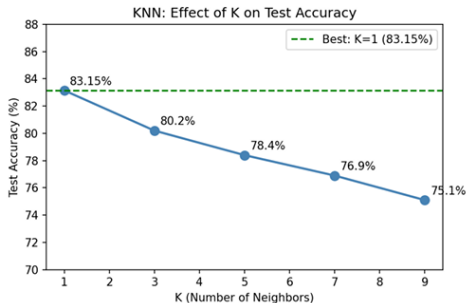
K-Nearest Neighbours (KNN)

How It Works

- Non-parametric, instance-based learning
- Majority vote of K nearest neighbours in PCA (450-dim) space
- No explicit training phase

K	Accuracy
1	83.15%
3	80.2%
5	78.4%
7	76.9%
9	75.1%

- ✓ Tight clusters in PCA space
- ✓ Single neighbour highly reliable
- ✗ Larger K causes cross-subject contamination



Gaussian Naive Bayes

Model

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^d P(x_i|y)$$

Each $P(x_i|y)$ modelled as $\mathcal{N}(\mu_{y,i}, \sigma_{y,i}^2)$.

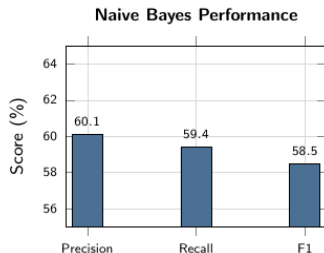
Result

Test Accuracy: **59.37%** Precision: $\approx 60.1\%$

Why It Struggles:

- × PCA components orthogonal but not independent
- × Gaussian fit poor for multi-modal illumination
- × Illumination creates correlated intensity patterns

Advantage: Extremely fast training, probabilistic output.



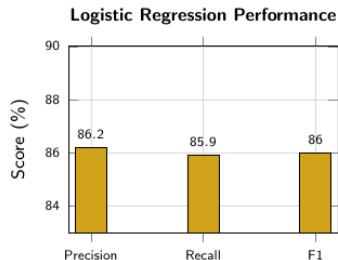
Logistic Regression

$$P(y|\mathbf{x}) = \frac{e^{\mathbf{w}_y^\top \mathbf{x}}}{\sum_k e^{\mathbf{w}_k^\top \mathbf{x}}}$$

Solver	L-BFGS
Regularization	$C = 10$ (L2)
Test Accuracy	85.93%
Macro F1	0.860

- ✓ PCA decorrelates features — linear boundaries work
- ✓ Directly optimizes cross-entropy loss
- ✓ Calibrated class probabilities
- ✗ Cannot capture non-linear illumination manifolds

ries



Support Vector Machine (SVM)

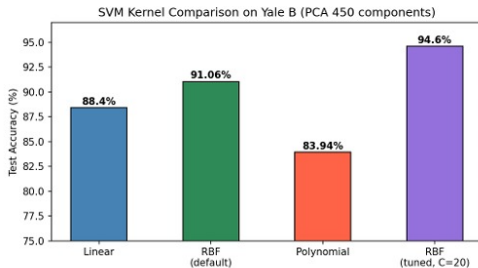
$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 \text{ s.t. } y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1$$

RBF: $K(\mathbf{x}, \mathbf{x}') = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$

Kernel	Acc.	F1
Linear	88.40%	0.884
Polynomial	83.94%	0.839
RBF (def.)	91.06%	0.910
RBF (tuned)	94.60%	0.945

Best: $C = 20$, $\gamma = \text{scale}$

- ✓ Illumination manifolds form convex cones (Basri & Jacobs)
- ✓ RBF maps curved manifolds to separable regions
- ✗ Linear kernel cannot handle curved manifolds



Random Forest Classifier

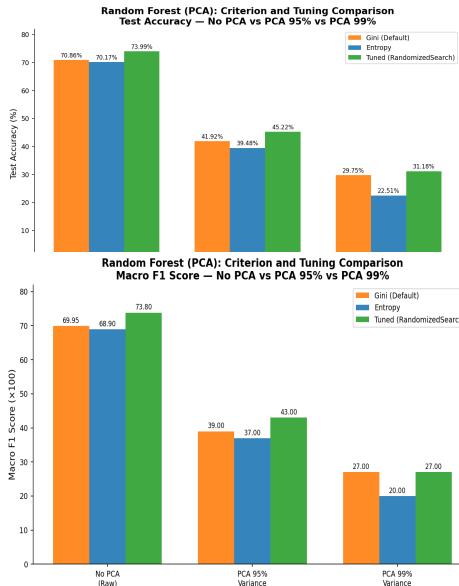
Model: Ensemble via Bagging

- Each tree trained on a bootstrap sample
- Random feature subset at each split ($\sqrt{d}$ features)
- Majority vote; OOB error for free validation

Variant	Accuracy
Gini (default)	70.86%
Entropy	70.17%
Tuned	73.99%

Limitation

Axis-aligned splits in PCA space are poorly aligned with non-linear illumination manifolds.



Classical ML: Summary

Classifier	Acc.	F1	Notes
Naive Bayes	59.37%	0.585	Violated assumptions
Random Forest	73.99%	0.740	Axis-aligned limits
KNN ($K = 1$)	83.15%	0.830	No training needed
Logistic Reg.	85.93%	0.860	Probabilistic output
SVM Linear	88.40%	0.884	Linear only
SVM RBF (def.)	91.06%	0.910	Non-linear bounds
SVM RBF (tuned)	94.60%	0.945	Best classical

Takeaway

PCA + SVM is an excellent baseline: competitive, fast to deploy, and interpretable.

Need for Deep Learning

SVM saturates at $\sim 94.6\%$. PCA is linear — misses non-linear illumination structure.

Motivation for Deep Learning

Limitations of Classical ML

- PCA is a linear transformation — misses non-linear illumination manifold structure
- Handcrafted features cannot adapt to data
- SVM saturates at $\sim 94\%$ — harder subjects still misclassified
- PCA discards spatial (pixel arrangement) information

Why Deep Learning?

CNNs learn hierarchical, non-linear features directly from pixels:

- Edges $\rightarrow$ textures $\rightarrow$ face parts
- Adapted to the exact recognition task
- No manual feature engineering required
- Spatial structure fully preserved

CNN from Scratch: Architecture

5-Block CNN Architecture

Block	Output Shape
Input (grayscale)	$1 \times 128 \times 128$
Conv1(32)+BN+ReLU+Pool	$32 \times 64 \times 64$
Conv2(64)+BN+ReLU+Pool	$64 \times 32 \times 32$
Conv3(128)+BN+ReLU+Pool	$128 \times 16 \times 16$
Conv4(256)+BN+ReLU+Pool	$256 \times 8 \times 8$
Conv5(512)+BN+ReLU+Pool	$512 \times 4 \times 4$
GlobalAvgPool	512
FC(256)→FC(128)→FC(28)	28

Result

Test Accuracy: **96.13%** F1: **0.9614** Params: 5.1M

Training Setup

- Optimizer: AdamW ($lr = 3 \times 10^{-4}$, $wd = 10^{-4}$)
- Scheduler: CosineAnnealingWarmRestarts
- Augmentation: flips, $\pm 15^\circ$ rotation, color jitter
- Dropout: 0.2–0.5 across blocks; Epochs: 30

Key Design Choices

- BatchNorm stabilizes training across blocks
- GlobalAvgPool reduces parameters vs. flattening
- Progressive channel doubling (32→512)

ResNet50: Transfer Learning

Residual Connections: $y = \mathcal{F}(x, \{W_i\}) + x$

- Solves vanishing gradient in 50-layer network
- Pretrained on ImageNet (1.28M images)
- Conv1 adapted to grayscale; final: Linear(2048→28)
- LR 10^{-5} , label smoothing $\varepsilon = 0.1$, 40 epochs

Result

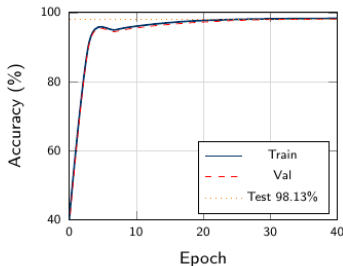
Test Acc: **98.13%** F1: **0.981** Params: 23.5M

Transfer Learning Power

91.53% val accuracy by **epoch 3**. CNN from scratch needed 10+ epochs.

k

ResNet50: Training & Validation Accuracy



FaceNet: Metric Learning

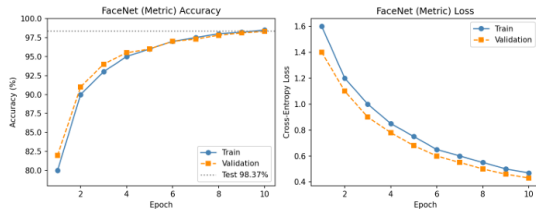
Triplet Loss:

$$\mathcal{L} = \sum_{a,p,n} [\|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha]_+$$

- 512-dim embedding; distance = face similarity
- Inception-ResNet-v1 pretrained on VGGFace2
- Last 20 layers fine-tuned; only 10 epochs

Shadow Recovery:

- 1 Gamma correction ($\gamma = 1.5$): brightens mid-tones
- 2 CLAHE: local contrast equalization



Result (Best Overall)

Accuracy: **98.37%** in only **10 epochs** F1:
0.984

FaceNet: Training Details & Per-Class Performance

Table: Training Configuration

Base Model	Inception-ResNet-v1
Pretrained On	VGGFace2
Fine-tuned Layers	Last 20
Classifier Head	BN→D(512)→Drop→D(28)
Epochs	10 Batch: 32
Preprocessing	Gamma + CLAHE

Table: Sample Per-Class Results

Subject	P	R	F1
yaleB11	1.00	1.00	1.00
yaleB12	1.00	0.97	0.98
yaleB13	1.00	1.00	1.00
yaleB15	0.96	0.96	0.96
yaleB17	0.93	0.98	0.95
All 28	~0.97+	~0.97+	~0.98

Key Insight

Epoch 1 val accuracy = 95.06% reflects the power of domain-specific face pretraining.

Deep Learning Models: Head-to-Head

Model	Acc.	F1	Params	Ep.
CNN (scratch)	96.13%	0.961	5.1M	30
ResNet50 (TL)	98.13%	0.981	23.5M	40
FaceNet	98.37%	0.984	28.0M	10

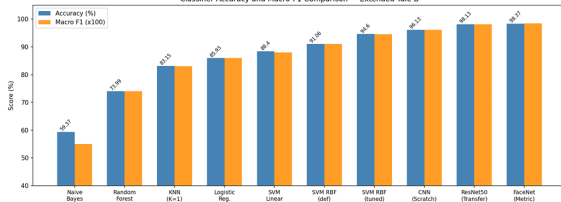
Model	Best For
CNN scratch	Low resources
ResNet50	Accuracy + speed
FaceNet	Max accuracy

Key Observations

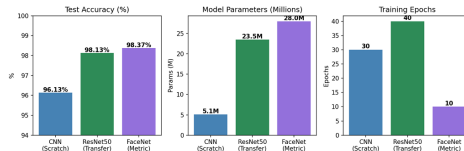
- CNN surpasses best classical (94.6%) via non-linear hierarchical features
- Domain-specific pretraining (FaceNet on faces) outperforms general pretraining (ResNet50/ImageNet)
- FaceNet: highest accuracy in just 10 epochs — most efficient when domain pretraining is available

Comparative Analysis: All 10 Classifiers

Classifier Accuracy and Macro-F1 Comparison — Extended Yale B



Deep Learning Models — Accuracy, Parameters, and Epochs



Accuracy Progression: NB (59.4%) < RF (74.0%) < KNN (83.2%) < LR (85.9%) < SVM-Lin (88.4%) < SVM-RBF (91.1%) < SVM-Tuned (94.6%) < CNN (96.1%) < ResNet (98.1%) < **FaceNet (98.4%)**

Complete Results Table

Type	Classifier	Accuracy	F1	Key Advantage
7*Classical	Naive Bayes	59.37%	0.585	Fastest training
	Random Forest	73.99%	0.740	OOB error estimate
	KNN ($K = 1$)	83.15%	0.830	No training phase
	Logistic Regression	85.93%	0.860	Probabilistic output
	SVM Linear	88.40%	0.884	Strong margins
	SVM RBF (default)	91.06%	0.910	Non-linear boundaries
	SVM RBF (tuned)	94.60%	0.945	Best classical
3*Deep	CNN from Scratch	96.13%	0.961	No feature engineering
	ResNet50 (TL)	98.13%	0.981	Rapid convergence
	FaceNet	98.37%	0.984	Best overall

Key Insights & Analysis

Why SVM Beats Other Classical Methods

- Margin maximization finds the most robust decision boundary
- RBF kernel maps illumination manifolds (convex cones) to linearly separable regions
- Tuned $C = 20$ balances margin width vs. misclassification penalty

Why Naive Bayes Performs Worst

- Conditional independence violated in PCA data
- Gaussian assumption ignores multi-modal illumination structure

Why FaceNet Wins

- Pretrained on millions of face images
- Triplet loss explicitly optimizes face similarity distances
- Shadow recovery (Gamma + CLAHE) handles Yale B's extreme illumination
- Domain-specific embedding outperforms general visual features

Deep vs. Classical Gap

Deep learning provides a consistent **$\sim 2-4\%$ gain** over the best classical methods by learning non-linear hierarchical features directly from pixels.

Conclusion & Future Work

Key Findings

- Accuracy hierarchy: $NB < RF < KNN < LR < SVM < CNN < ResNet < \text{FaceNet}$
- $SVM + PCA$ = best classical model (94.6%), preferred when GPU unavailable
- FaceNet = best overall (98.37%) in only 10 epochs
- Shadow recovery (CLAHE) critical for extreme illumination

Final Takeaway

PCA + SVM: excellent baseline when resources are constrained.

FaceNet: clear choice when accuracy is the primary objective.

Future Work

- ViT / Swin Transformers for global context modeling
- Ensemble of CNN + SVM for hybrid approach
- Grad-CAM for explainability — which face regions matter?
- Cross-dataset evaluation (LFW, MS-Celeb)
- Robust augmentation with synthetic extreme lighting
- Mean $\pm$ std over multiple seeds for rigorous evaluation

References

- [1] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," Proc. CVPR 2015. arXiv:1503.03832
- [2] A. S. Georghiades, P. N. Belhumeur, and D. J. Kriegman, "Extended Yale Face Database B," Yale Vision Group. Available: <http://vision.ucsd.edu/leekc/ExtYaleDatabase/ExtYaleB.html>
- [3] scikit-learn Developers, "scikit-learn: Machine Learning in Python," v1.4.
Available: https://scikit-learn.org/stable/user_guide.html
- [4] PyTorch Contributors, "PyTorch Documentation – torchvision.models," Meta AI.
Available: <https://pytorch.org/vision/stable/models.html>
- [5] A. Karpathy and L. Fei-Fei, "CS231n: Convolutional Neural Networks for Visual Recognition," Stanford University. Available: <https://cs231n.github.io>

Thank You!

Questions & Discussion

N. Rushil Krishna Sai	MC24BT011
N. Suhas	CS24BT019
P. Sanjana	CS24BT002
Trisham Bepari	CS24BT022
Ashish Kumar	MC24BT028

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad, 2024